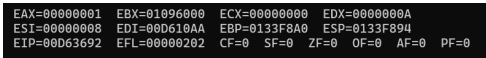
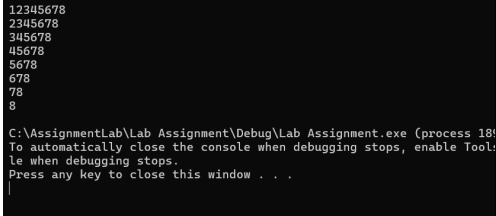


25006180 CSA Lab Assignment

No	Code	Output
1.	<pre> INCLUDE Irvine32.inc .data myArray WORD 0, 2, 5, 9, 10 gapsSum DWORD 0 .code main PROC mov esi, 0 ; index = 0 mov ecx, 4 ; number of gaps = n-1 mov edx, 0 ; sum = 0 L1: mov ax, myArray[esi + 2] ; next element sub ax, myArray[esi] ; next - current movzx eax, ax ; extend to 32-bit add edx, eax ; sum += gap add esi, 2 ; move to next WORD loop L1 mov gapsSum, edx call DumpRegs exit main ENDP END main </pre>	
2.	<pre> INCLUDE Irvine32.inc .data myArray WORD 0, 2, 5, 9, 10 gapsSum DWORD 0 .code main PROC mov esi, 0 ; index = 0 mov ecx, 4 ; number of gaps = n-1 mov edx, 0 ; sum = 0 </pre>	

	<pre> L1: mov ax, myArray[esi + 2] ; next element sub ax, myArray[esi] ; next - current movzx eax, ax ; extend to 32-bit add edx, eax ; sum += gap add esi, 2 ; move to next WORD loop L1 mov gapsSum, edx call DumpRegs exit main ENDP END main </pre>	
3.	<pre> INCLUDE Irvine32.inc .data prompt BYTE "Enter 32-bit integer : ", 0 result BYTE "The sum of 32-bit integers is : ", 0 myArray DWORD 3 DUP(?) ; Allocate memory buffer for 3 uninitialized DWORDs .code main PROC ; --- Step 1: Read inputs from user and store into Array --- mov esi, 0 ; Index pointer starts at 0 ; Integer 1 mov edx, OFFSET prompt call WriteString call ReadInt mov myArray[esi], eax add esi, 4 ; Move pointer forward by 4 bytes (DWORD size) ; Integer 2 call WriteString call ReadInt mov myArray[esi], eax </pre>	<pre> Enter 32-bit integer : 1 Enter 32-bit integer : 2 Enter 32-bit integer : 2 The sum of 32-bit integers is : +5 C:\AssignmentLab\Lab Assignment\Debug\Lab Assign To automatically close the console when debuggi le when debugging stops. Press any key to close this window . . . </pre>

	<pre> add esi, 4 ; Integer 3 call WriteString call ReadInt mov myArray[esi], eax ; --- Step 2: Sum the stored array values --- mov eax, 0 ; Reset accumulation register to 0 mov ecx, 3 ; Loop counter set to 3 elements mov esi, 0 ; Reset index pointer to start of array SumLoop: add eax, myArray[esi] add esi, 4 ; Advance pointer to next DWORD element loop SumLoop ; --- Step 3: Print result --- mov edx, OFFSET result call WriteString mov ebx, TYPE myArray ; Set up formatting for positive sign display if needed call WriteDec ; Prints final sum stored inside EAX call Crlf exit main ENDP END main </pre>	
4.	<pre> INCLUDE Irvine32.inc .data promptMark BYTE "Enter mark (0-100): ", 0 outputGrade BYTE "Grade: ", 0 .code main PROC ; Prompt user for mark input </pre>	<pre> Enter mark (0-100): 12 Grade: F C:\AssignmentLab\Lab Assignment\Debug\Lab Assignr To automatically close the console when debuggin le when debugging stops. Press any key to close this window . . . </pre>

	<pre> mov edx, OFFSET promptMark call WriteString call ReadInt ; Input integer is automatically saved into EAX register ; Display grade prefix label mov edx, OFFSET outputGrade call WriteString ; Call our custom grading procedure call DetermineGrade call Crlf exit main ENDP ; ----- -- ; DetermineGrade PROC ; Receives: EAX = integer score (0-100) ; Displays: Single capital letter matching the grade scale ; ----- -- DetermineGrade PROC cmp eax, 90 jge GradeA ; Jump if score >= 90 cmp eax, 80 jge GradeB ; Jump if score >= 80 cmp eax, 70 jge GradeC ; Jump if score >= 70 cmp eax, 60 jge GradeD ; Jump if score >= 60 jmp GradeF ; Default case if score < 60 GradeA: mov al, 'A' jmp PrintChar </pre>	
--	--	--

	<pre>GradeB: mov al, 'B' jmp PrintChar GradeC: mov al, 'C' jmp PrintChar GradeD: mov al, 'D' jmp PrintChar GradeF: mov al, 'F' PrintChar: call WriteChar ; Output the character ; currently inside the AL register ret DetermineGrade ENDP END main</pre>	
--	---	--